



SMART CONTRACT **AUDIT REPORT**

Argopool Smart Contract

JUNE 2026

Contents

1. EXECUTIVE SUMMARY	4
1.1 Methodology	4
2. FINDINGS OVERVIEW	7
2.1 Project Info And Contract Address	7
2.2 Summary	7
2.3 Key Findings	8
3. DETAILED DESCRIPTION OF FINDINGS	9
3.1 Inflated matchedUsd accounting allows recycled deposits to distort fee-share tiers	9
3.2 Mutable prediction inputs can mislead users into prefunding an unrecoverable address .	11
3.3 Spot-price accounting can inflate fee-share tiers for pool-priced deposits	13
3.4 Silent balanceOf fallback can misaccount collected fees and principal	16
3.5 positionOriginalUsd is recorded from desired mint amounts instead of actual deposited amounts	18
3.6 Stale operator privileges survive admin rotation and preserve execution control	20
3.7 Nominal deposit amounts can overstate fee-tier accounting for supported non-standard tokens	22
3.8 Unbounded fee-share bps can block fee collection and position closure	23
3.9 Manager external updates can make existing ContrAccount instances incompatible with managed operations	24
3.10 ContrAccount admin is stored but never used	26
3.11 zapCreatePositionETH V1 does not ensure swap.amountIn is bounded by ethAmount .	27
4. CONCLUSION	28
5.1 Basic Coding Assessment	29
5.1.1 Apply Verification Control	29
5.1.2 Authorization Access Control	29
5.1.3 Forged Transfer Vulnerability	29
5.1.4 Transaction Rollback Attack	30
5.1.5 Transaction Block Stuffing Attack	30
5.1.6 Soft Fail Attack Assessment	30
5.1.7 Hard Fail Attack Assessment	31
5.1.8 Abnormal Memo Assessment	31
5.1.9 Abnormal Resource Consumption	31
5.1.10 Random Number Security	32

5.2 Advanced Code Scrutiny	32
5.2.1 Cryptography Security	32
5.2.2 Account Permission Control	32
5.2.3 Malicious Code Behavior	33
5.2.4 Sensitive Information Disclosure	33
5.2.5 System API	33
6. DISCLAIMER	34
7. REFERENCES	35
8. About Exvul Security	36

1. EXECUTIVE SUMMARY

ExVul Web3 Security was engaged by **Argopool** to review smart contract implementation. The assessment was conducted in accordance with our systematic approach to evaluate potential security issues based upon customer requirement. The report provides detailed recommendations to resolve the issue and provide additional suggestions or recommendations for improvement.

The outcome of the assessment outlined in chapter 3 provides the system owners a full description of the vulnerabilities identified, the associated risk rating for each vulnerability, and detailed recommendations that will resolve the underlying technical issue.

1.1 Methodology

To standardize the evaluation, we define the following terminology based on OWASP Risk Rating Methodology [10] which is the gold standard in risk assessment using the following risk models:

- **Likelihood:** represents how likely a particular vulnerability is to be uncovered and exploited in the wild.
- **Impact:** measures the technical loss and business damage of a successful attack.
- **Severity:** determine the overall criticality of the risk.

Likelihood can be: High, Medium and Low and impact are categorized into: High, Medium, Low, Informational. Severity is determined by likelihood and impact and can be classified into five categories accordingly: Critical, High, Medium, Low, Informational shown in table 1.1.

		Informational	Low	Medium	High
Likelihood	High	INFO	MEDIUM	HIGH	CRITICAL
	Medium	INFO	LOW	MEDIUM	HIGH
	Low	INFO	LOW	LOW	MEDIUM
		IMPACT			

Table 1.1 Overall Risk Severity

To evaluate the risk, we will be going through a list of items, and each would be labelled with a severity category. The audit was performed with a systematic approach guided by a comprehensive assessment list carefully designed to identify known and impactful security issues. If our tool or analysis does not identify any issue, the contract can be considered safe regarding the assessed item. For any discovered issue, we might further deploy contracts on our private test environment and run tests to confirm the findings. If necessary, we would additionally build a PoC to demonstrate the possibility of exploitation. The concrete list of check items is shown in Table 1.2.

- **Basic Coding Bugs:** We first statically analyze given smart contracts with our proprietary static code analyzer for known coding bugs, and then manually verify all the issues found by our tool.
- **Code and business security testing:** We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.
- **Additional Recommendations:** We also provide additional suggestions regarding the coding and development of smart contracts from the perspective of proven programming practices.

Category	Assessment Item
Basic Coding Assessment	• Apply Verification Control • Authorization Access Control • Forged Transfer Vulnerability • Forged Transfer Notification • Numeric Overflow • Transaction Rollback Attack • Transaction Block Stuffing Attack • Soft Fail Attack • Hard Fail Attack • Abnormal Memo • Abnormal Resource Consumption • Secure Random Number

Advanced Source Code Scrutiny	• Asset Security • Cryptography Security • Business Logic Review • Source Code Functional Verification • Account Authorization Control • Sensitive Information Disclosure • Circuit Breaker • Blacklist Control • System API Call Analysis • Contract Deployment Consistency Check • Abnormal Resource Consumption
Additional Recommendations	• Semantic Consistency Checks • Following Other Best Practices

Table 1.2: The Full List of Assessment Items

To better describe each issue we identified, we categorize the findings with Common Weakness Enumeration (CWE-699) [14], which is a community-developed list of software weakness types to better delineate and organize weaknesses around concepts frequently encountered in software development.

2. FINDINGS OVERVIEW

2.1 Project Info And Contract Address

Project Name	Audit Time	Language
Argopool	2026-06	Solidity

File Hash

ContrAccount.sol: CF024D8085B0D54A7A484F5E2F54ED9B

Factory.sol: 85B9190EB1B2118B7A8DA5A2E2E27706

Manager.sol: E927D5A41A5E5E420ECFB90CAC67853D

2.2 Summary

Severity	Found
CRITICAL	0
HIGH	0
MEDIUM	5
LOW	4
INFO	2



2.3 Key Findings

Severity	Findings Title	Status
MEDIUM	Inflated matchedUsd accounting allows recycled deposits to distort fee-share tiers	Acknowledged
MEDIUM	Mutable prediction inputs can mislead users into prefunding an unrecoverable address	Acknowledged
MEDIUM	Spot-price accounting can inflate fee-share tiers for pool-priced deposits	Acknowledged
MEDIUM	Silent balanceOf fallback can misaccount collected fees and principal	Fixed
MEDIUM	positionOriginalUsd is recorded from desired mint amounts instead of actual deposited amounts	Acknowledged
LOW	Stale operator privileges survive admin rotation and preserve execution control	Fixed
LOW	Nominal deposit amounts can overstate fee-tier accounting for supported non-standard tokens	Acknowledged
LOW	Unbounded fee-share bps can block fee collection and position closure	Acknowledged
LOW	Manager external updates can make existing ContrAccount instances incompatible with managed operations	Fixed
INFO	ContrAccount admin is stored but never used	Fixed
INFO	zapCreatePositionETH V1 does not ensure swap.amountIn is bounded by ethAmount	Acknowledged

Table 2.3: Key Audit Findings

3. DETAILED DESCRIPTION OF FINDINGS

3.1 Inflated matchedUsd accounting allows recycled deposits to distort fee-share tiers

SEVERITY:**MEDIUM****STATUS:****Acknowledged****PATH:**

ContrAccount.sol

DESCRIPTION:

`depositToken()` and `depositNative()` increase `matchedUsd` immediately after quoting the deposited asset in USD, while the owner withdrawal paths transfer assets without reducing the same accounting variable.

```
function depositToken(address token, uint256 amount)
    external
    onlyOwner
    nonReentrant
{
    ...
    uint256 usdValue =
        IManagerQuote(manager).quoteTokenToUsd(token, amount);

    matchedUsd += usdValue;
}

function withdrawAllToken(address token)
    external
    onlyOwner
    nonReentrant
{
    uint256 bal = IERC20(token).balanceOf(address(this));
    token.call(...);
    emit TokenWithdrawn(token, bal);
}
```

Fee distribution derives the user share directly from `feeRule.getUserFeeBps(matchedUsd)`,

so recycled deposits can distort the account's tiering state.

```
uint16 bps = IFeeRule(feeRule).getUserFeeBps(matchedUsd);
feeAmount0ToOwner = (delta0 * bps) / 10000;
feeAmount1ToOwner = (delta1 * bps) / 10000;
```

IMPACT:

The same capital can be deposited, withdrawn, and redeposited repeatedly to accumulate an arbitrarily large `matchedUsd` value that no longer reflects capital still committed to the account or LP lifecycle.

RECOMMENDATIONS:

Decrease `matchedUsd` whenever the owner withdraws unmatched assets from the account.

```
function withdrawToken(address token, uint256 amount) external onlyOwner
    nonReentrant {
    ...
+   uint256 usdValue = IManagerQuote(manager).quoteTokenToUsd(token,
+   amount);
+   if (usdValue >= matchedUsd) {
+       matchedUsd = 0;
+   } else {
+       matchedUsd -= usdValue;
+   }
    emit TokenWithdrawn(token, amount);
}

function withdrawETH(uint256 amount) external onlyOwner nonReentrant {
    ...
+   uint256 usdValue = IManagerQuote(manager).quoteNativeToUsd(amount);
+   if (usdValue >= matchedUsd) {
+       matchedUsd = 0;
+   } else {
+       matchedUsd -= usdValue;
+   }
    emit ETHWithdrawn(amount);
}
```

3.2 Mutable prediction inputs can mislead users into prefunding an unrecoverable address

SEVERITY:**MEDIUM****STATUS:****Acknowledged****PATH:**

Factory.sol

DESCRIPTION:

`predictContrAccount()` presents a deterministic CREATE2 account preview, but the returned address is derived not only from `owner` and `salt`, but also from mutable arguments such as `manager`, `admin`, `positionManager`, `swapRouter`, `feeRecipient`, and `feeRule`.

```
function createContrAccount() external payable returns (address contr) {
    ...
    contr = address(new ContrAccount{salt: salt}(
        msg.sender,
        manager,
        admin,
        positionManager,
        swapRouter,
        feeRecipient,
        feeRule
    ));
}

function predictContrAccount(address owner) external view returns
(address predicted) {
    bytes memory bytecode = abi.encodePacked(
        type(ContrAccount).creationCode,
        abi.encode(owner, manager, admin, positionManager, swapRouter,
        feeRecipient, feeRule)
    );
    ...
}
```

Since the admin can change those values before `createContrAccount()` is called, the previously

predicted address can become stale while still looking authoritative to users or frontends that treat it as a prefunding target.

IMPACT:

A user can prefund a predicted account address that later becomes undeployable after an admin configuration change. Once the user creates the new account at the updated address, the assets sent to the stale predicted address can become permanently stranded.

RECOMMENDATIONS:

Remove `predictContrAccount()` and do not expose its output as a prefunding target.

3.3 Spot-price accounting can inflate fee-share tiers for pool-priced deposits

SEVERITY:

MEDIUM

STATUS:

Acknowledged

PATH:`ContrAccount.sol\Manager.sol`**DESCRIPTION:**

`depositToken()` permanently increases `matchedUsd` using `Manager.quoteTokenToUsd()`, while `_quoteTokenToUsd()` prices supported non-USD tokens from the current Uniswap V3 `slot0()` value. `slot0()` is an instantaneous pool price, not a TWAP or trusted oracle, so a temporarily manipulated token/USD pool can be written into long-lived fee-tier accounting.

```
function depositToken(address token, uint256 amount) external onlyOwner
    nonReentrant {
        ...
        uint256 usdValue = IManagerQuote(manager).quoteTokenToUsd(token,
            amount);
        matchedUsd += usdValue;
    }

function _quoteTokenToUsd(address token, uint256 amount) internal view
    returns (uint256) {
        ...
        (uint160 sqrtPriceX96,,,,,) = uniPool.slot0();
        require(sqrtPriceX96 > 0, 'BAD_PRICE');
        ...
    }
```

Because fee sharing depends on `matchedUsd`, the manipulated accounting can qualify an account for a higher fee-share tier and reduce protocol fee revenue.

IMPACT:

A user can deposit while a configured token/USD pool spot price is temporarily inflated, causing `matchedUsd` to exceed the real deposited value and increasing later fee-share entitlement.

RECOMMENDATIONS:

Do not use Uniswap V3 `slot0()` as a long term accounting oracle.

```
+import '@uniswap/v3-periphery/contracts/libraries/OracleLibrary.sol';
+
contract Manager {
+   uint32 public twapPeriod = 1800;
+
+   function setTwapPeriod(uint32 newTwapPeriod) external onlyAdmin {
+       require(newTwapPeriod >= 300, 'TWAP_TOO_SHORT');
+       twapPeriod = newTwapPeriod;
+   }

    function _quoteTokenToUsd(address token, uint256 amount)
        internal
        view
        returns (uint256)
    {
        ...
-       (uint160 sqrtPriceX96,,,,,) = uniPool.slot0();
-       require(sqrtPriceX96 > 0, 'BAD_PRICE');
-
-       uint256 usdRaw;
-
-       if (token0 == token) {
-           uint256 step1 = FullMath.mulDiv(amount,
uint256(sqrtPriceX96), Q96);
-           usdRaw = FullMath.mulDiv(step1, uint256(sqrtPriceX96), Q96);
-       } else {
-           uint256 step1 = FullMath.mulDiv(amount, Q96,
uint256(sqrtPriceX96));
-           usdRaw = FullMath.mulDiv(step1, Q96, uint256(sqrtPriceX96));
-       }
+       (int24 arithmeticMeanTick, uint128 harmonicMeanLiquidity) =
+           OracleLibrary.consult(pool, twapPeriod);
+       require(harmonicMeanLiquidity > 0, 'NO_TWAP_LIQUIDITY');
+       require(amount <= type(uint128).max, 'AMOUNT_TOO_LARGE');
+
+       uint256 usdRaw = OracleLibrary.getQuoteAtTick(
+           arithmeticMeanTick,
+           uint128(amount),
+           token,
```

```
+         usdToken
+         );
    return _normalizeToUsdDecimals(usdRaw, usdTokenDecimals);
}
}
```

3.4 Silent balanceOf fallback can misaccount collected fees and principal

SEVERITY:

MEDIUM

STATUS:

Fixed

PATH:

ContrAccount.sol

DESCRIPTION:

`_safeBalanceOf()` returns 0 when `balanceOf()` reverts or returns malformed data, instead of reverting.

```
function _safeBalanceOf(address token, address account) internal view
    returns (uint256) {
    (bool ok, bytes memory data) =

    token.staticcall(abi.encodeWithSelector(IERC20.balanceOf.selector,
        account));

    if (!ok || data.length < 32) {
        return 0;
    }

    return abi.decode(data, (uint256));
}
```

That helper is used to compute the actual amount collected by balance delta in both `_collectAndSplitWithAmounts()` and `_collectPrincipalOnlyWithAmounts()`. If settlement succeeds while both balance reads are silently coerced to zero, collected tokens remain in the user-controlled account and protocol accounting under-collects.

IMPACT:

Collected tokens can remain in `ContrAccount`, and the owner can later withdraw them through owner withdrawal paths, causing protocol under-collection.

RECOMMENDATIONS:

Do not silently map failed `balanceOf()` calls to zero on settlement-critical paths; revert instead.

```
function _safeBalanceOf(address token, address account) internal view
    returns (uint256) {
    (bool ok, bytes memory data) =

    token.staticcall(abi.encodeWithSelector(IERC20.balanceOf.selector,
    account));

+   require(ok, 'BALANCEOF_CALL_FAILED');
+   require(data.length >= 32, 'BALANCEOF_BAD_RETURN');
    return abi.decode(data, (uint256));
}
```

3.5 positionOriginalUsd is recorded from desired mint amounts instead of actual deposited amounts

SEVERITY:**MEDIUM****STATUS:****Acknowledged****PATH:**`Manager.sol\ContrAccount.sol`**DESCRIPTION:**

`Manager.mintPosition()` and the zap-based position creation paths calculate `originalUsd` from the requested `amount0Desired` and `amount1Desired` before the Uniswap V3 mint is executed.

```
function mintPosition(address contrAccount, MintParams calldata p, bool
    baseIsToken0)
    external
    onlyOperator
    returns (uint256 tokenId, uint128 liquidity, uint256 amount0, uint256
        amount1)
{
    uint256 originalUsd =
        _quoteTokenToUsd(p.token0, p.amount0Desired) +
        _quoteTokenToUsd(p.token1, p.amount1Desired);

    (tokenId, liquidity, amount0, amount1) = _mintPosition(contrAccount,
        p);
    IContrAccount(contrAccount).registerPosition(tokenId, originalUsd,
        baseIsToken0);
}
```

However, in Uniswap V3, `amount0Desired` and `amount1Desired` are maximum input amounts, not the actual amounts consumed by the minted position. `ContrAccount.closePosition()` later decreases `matchedUsd` using the stored `positionOriginalUsd[tokenId]`, so the unwind path also depends on this inflated value.

IMPACT:

The account records and later unwinds a USD value based on intended mint inputs rather than the real principal actually deployed into the Uniswap V3 position.

RECOMMENDATIONS:

Modify `Manager.mintPosition()` and all zap-based position creation functions so that `originalUsd` is calculated after `_mintPosition()` returns, using the actual `amount0` and `amount1` values returned by Uniswap V3.

3.6 Stale operator privileges survive admin rotation and preserve execution control

SEVERITY:

LOW

STATUS:

Fixed

PATH:

Manager.sol

DESCRIPTION:

The constructor grants the initial `admin` full `operator` execution rights through `isOperator[_admin] = true` and `_addOperator(_admin)`, but `setAdmin()` later updates only the `admin` storage slot.

```
constructor(address _admin, address _positionManager, address
    _swapRouter) {
    ...
    admin = _admin;
    positionManager = _positionManager;
    swapRouter = _swapRouter;

    isOperator[_admin] = true;
    _addOperator(_admin);
    emit OperatorUpdated(_admin, true);
}

function setAdmin(address newAdmin) external onlyAdmin {
    require(newAdmin != address(0), 'ZERO_ADMIN');
    emit AdminUpdated(admin, newAdmin);
    admin = newAdmin;
}
```

Sensitive portfolio actions such as `mintPosition`, `swapExactInputSingle`, and `closePosition` continue to trust the `onlyOperator` role, so a stale operator mapping preserves execution control after admin rotation.

IMPACT:

Any governance rotation that executes `setAdmin()` without an immediate matching `setOperator()`

update leaves the former admin with a live `onlyOperator` privilege set.

RECOMMENDATIONS:

Revoke the old admin operator rights and grant the new admin the same execution role atomically.

```
function setAdmin(address newAdmin) external onlyAdmin {
    require(newAdmin != address(0), 'ZERO_ADMIN ');
+   address oldAdmin = admin;
+   if (oldAdmin != newAdmin) {
+       isOperator[oldAdmin] = false;
+       _removeOperator(oldAdmin);
+       isOperator[newAdmin] = true;
+       _addOperator(newAdmin);
+       emit OperatorUpdated(oldAdmin, false);
+       emit OperatorUpdated(newAdmin, true);
+   }
    emit AdminUpdated(oldAdmin, newAdmin);
    admin = newAdmin;
}
```

3.7 Nominal deposit amounts can overstate fee-tier accounting for supported non-standard tokens

SEVERITY:

LOW

STATUS:

Acknowledged

PATH:`ContrAccount.sol\Manager.sol`**DESCRIPTION:**

`depositToken()` credits `matchedUsd` from the user-supplied `amount` after calling `quoteTokenToUsd(token, amount)`, but never checks how many tokens were actually received by the account. If any supported token is fee-on-transfer, taxed, rebasing, or otherwise transfers less than `amount`, `matchedUsd` is credited against the nominal input instead of the actual received balance.

```
function depositToken(address token, uint256 amount)
    external
    onlyOwner
    nonReentrant
{
    ...
    emit DepositToken(owner, address(this), token, amount);
    uint256 usdValue = IManagerQuote(manager).quoteTokenToUsd(token,
        amount);
    matchedUsd += usdValue;
}
```

IMPACT:

For supported non-standard tokens, `matchedUsd` can become larger than the account's real received value and slightly overstate later fee-tier eligibility.

RECOMMENDATIONS:

No support for non-standard tokens.

3.8 Unbounded fee-share bps can block fee collection and position closure

SEVERITY:

LOW

STATUS:

Acknowledged

PATH:

ContrAccount.sol

DESCRIPTION:

`_collectAndSplitWithAmounts()` trusts `feeRule.getUserFeeBps(matchedUsd)` as a valid basis-point percentage, but `ContrAccount` never enforces `bps <= 10000`.

```
uint16 bps = IFeeRule(feeRule).getUserFeeBps(matchedUsd);
feeAmount0ToOwner = (delta0 * bps) / 10000;
feeAmount1ToOwner = (delta1 * bps) / 10000;
protocolFee0 = delta0 - feeAmount0ToOwner;
protocolFee1 = delta1 - feeAmount1ToOwner;
```

Both `collectFees()` and `closePosition()` depend on this same split path before completing normal settlement.

IMPACT:

A bad `feeRule` return value can make fee collection and position closure revert for active positions, blocking normal fee settlement and exit until the rule is corrected.

RECOMMENDATIONS:

Clamp or validate the returned basis points before using them in fee split calculations.

```
uint16 bps = IFeeRule(feeRule).getUserFeeBps(matchedUsd);
+if (bps > 10000) {
+    bps = 10000;
+}
feeAmount0ToOwner = (delta0 * bps) / 10000;
feeAmount1ToOwner = (delta1 * bps) / 10000;
```

3.9 Manager external updates can make existing ContrAccount instances incompatible with managed operations

SEVERITY:

LOW

STATUS:

Fixed

PATH:

Manager.sol\ContrAccount.sol

DESCRIPTION:

[Manager](#) allows the admin to update the active Uniswap V3 [positionManager](#) and [swapRouter](#), while each [ContrAccount](#) stores its own copies of those addresses at construction time.

```
function setExternals(address pm, address router) external onlyAdmin {
    require(pm != address(0), 'ZERO_PM ');
    require(router != address(0), 'ZERO_ROUTER ');
    positionManager = pm;
    swapRouter = router;
}

constructor(
    address _owner,
    address _manager,
    address _admin,
    address _positionManager,
    address _swapRouter,
    ...
) {
    ...
    positionManager = _positionManager;
    swapRouter = _swapRouter;
}
```

After an external address update, old accounts can reject managed actions that rely on the new manager-side addresses because account-side authorization still points to the older endpoints.

IMPACT:

Existing accounts can become incompatible with managed operations after an external address update. Operator-driven flows may revert with `INVALID_TARGET` or `INVALID_SPENDER` when executed against old accounts.

RECOMMENDATIONS:

Use one consistent upgrade model across `Manager` and `ContrAccount`. Either keep external dependencies immutable per account, or add an explicit synchronized migration path that updates account-side trusted endpoints before the manager switches globals.

3.10 ContrAccount admin is stored but never used

SEVERITY: INFO**STATUS:** Fixed**PATH:**`ContrAccount.sol`**DESCRIPTION:**

`ContrAccount` receives an `_admin` argument during construction and stores it in the `admin` state variable, but no later execution path uses that value for authorization or configuration.

```
constructor(  
    address _owner,  
    address _manager,  
    address _admin,  
    address _positionManager,  
    address _swapRouter,  
    address _feeRecipient,  
    address _feeRule  
) {  
    ...  
    admin = _admin;  
    ...  
}
```

IMPACT:

This unused state variable can confuse integrators and reviewers by implying that the protocol admin has some direct authority over `ContrAccount` instances when no such authority is implemented. It also increases deployment and storage complexity without providing functional benefit.

3.11 zapCreatePositionETH V1 does not ensure swap.amountIn is bounded by ethAmount

SEVERITY: INFO**STATUS:** Acknowledged**PATH:**`Manager.sol`**DESCRIPTION:**

`Manager.zapCreatePositionETH()` wraps `z.ethAmount` into WETH before optionally executing a swap, but the V1 ETH zap path does not check that `z.swap.amountIn` $\leq$ `z.ethAmount`.

```
function zapCreatePositionETH(address contrAccount, ZapETHParams calldata
    z, bool baseIsToken0)
    external
    onlyOperator
{
    ...
    IContrAccount(contrAccount).wrapETH(z.weth, z.ethAmount);
    ...
    if (z.swap.amountIn > 0) {
        require(z.swap.tokenIn == z.weth, 'NATIVE_SWAP_IN_NOT_WETH');
        ...
    }
}
```

As a result, the V1 ETH zap path can consume WETH that was already present in the `ContrAccount`, not only the WETH produced by the current `ethAmount`.

IMPACT:

This does not create a new direct asset-transfer capability because the caller must be an authorized operator, but it makes the V1 ETH zap semantics less precise than expected: a zap request that appears to be funded by `ethAmount` may also consume pre-existing WETH held by the account.

4. CONCLUSION

In this audit, we thoroughly analyzed **Argopool** smart contract implementation. The problems found are described and explained in detail in Section 3. The problems found in the audit have been communicated to the project leader. We therefore consider the audit result to be **PASSED**.

To improve this report, we greatly appreciate any constructive feedbacks or suggestions, on our methodology, audit findings, or potential gaps in scope/coverage.

5. APPENDIX

5.1 Basic Coding Assessment**5.1.1 Apply Verification Control**

Description	The security of apply verification
Result	Not found
Severity	CRITICAL

5.1.2 Authorization Access Control

Description	Permission checks for external integral functions
Result	Not found
Severity	CRITICAL

5.1.3 Forged Transfer Vulnerability

Description	Assess whether there is a forged transfer notification vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.4 Transaction Rollback Attack

Description	Assess whether there is transaction rollback attack vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.5 Transaction Block Stuffing Attack

Description	Assess whether there is transaction blocking attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.6 Soft Fail Attack Assessment

Description	Assess whether there is soft fail attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.7 Hard Fail Attack Assessment

Description	Examine for hard fail attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.8 Abnormal Memo Assessment

Description	Assess whether there is abnormal memo vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.9 Abnormal Resource Consumption

Description	Examine whether abnormal resource consumption in contract processing
Result	Not found
Severity	CRITICAL

5.1.10 Random Number Security

Description	Examine whether the code uses insecure random number
Result	Not found
Severity	CRITICAL

5.2 Advanced Code Scrutiny

5.2.1 Cryptography Security

Description	Examine for weakness in cryptograph implementation
Result	Not found
Severity	HIGH

5.2.2 Account Permission Control

Description	Examine permission control issue in the contract
Result	Not found
Severity	MEDIUM

5.2.3 Malicious Code Behavior

Description	Examine whether sensitive behavior present in the code
Result	Not found
Severity	MEDIUM

5.2.4 Sensitive Information Disclosure

Description	Examine whether sensitive information disclosure issue present in the code
Result	Not found
Severity	MEDIUM

5.2.5 System API

Description	Examine whether system API application issue present in the code
Result	Not found
Severity	LOW

6. DISCLAIMER

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to the Company in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes without ExVul's prior written consent.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts ExVul to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. ExVul's position is that each company and individual are responsible for their own due diligence and continuous security. ExVul's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

7. REFERENCES

- [1] MITRE. CWE-191: Integer Underflow (Wrap or Wraparound). <https://cwe.mitre.org/data/definitions/191.html>.
 - [2] MITRE. CWE-197: Numeric Truncation Error. <https://cwe.mitre.org/data/definitions/197.html>.
 - [3] MITRE. CWE-400: Uncontrolled Resource Consumption. <https://cwe.mitre.org/data/definitions/400.html>.
 - [4] MITRE. CWE-440: Expected Behavior Violation. <https://cwe.mitre.org/data/definitions/440.html>.
 - [5] MITRE. CWE-684: Protection Mechanism Failure. <https://cwe.mitre.org/data/definitions/693.html>.
 - [6] MITRE. CWE CATEGORY: 7PK - Security Features. <https://cwe.mitre.org/data/definitions/254.html>.
 - [7] MITRE. CWE CATEGORY: Behavioral Problems. <https://cwe.mitre.org/data/definitions/438.html>.
 - [8] MITRE. CWE CATEGORY: Numeric Errors. <https://cwe.mitre.org/data/definitions/189.html>.
 - [9] MITRE. CWE CATEGORY: Resource Management Errors. <https://cwe.mitre.org/data/definitions/399.html>.
 - [10] OWASP. Risk Rating Methodology. https://www.owasp.org/index.php/OWASP_Risk_Rating_Methodology
-

8. About Exvul Security

Premier Security for the Web3 Ecosystem

ExVul is a premier Web3 security firm committed to forging a secure and trustworthy decentralized ecosystem. Our elite team consists of security veterans from world-leading technology and blockchain security firms, including Huawei, YBB Captical, Qihoo 360, Amber, ByteDance, MoveBit, and PeckShield. Team member Nolan is ranked as a top-40 whitehat on Immunefi and is the platform's sole All-Star in the APAC region.

Our expertise covers the full spectrum of Web3 security. We conduct **meticulous smart contract audits**, having fortified thousands of projects on chains like Evm, Solana, Aptos, Sui etc. Our **Blockchain Protocol Audits** secure the core infrastructure of L1/L2 by uncovering deep-seated vulnerabilities. We also offer **comprehensive wallet audits** to protect user assets and provide **proactive web3 pentest**, enabling partners to neutralize threats before they strike.

Trusted by industry leaders, ExVul is the security partner for **OKX, Bitget, Cobo, Infini, Stacks, Aptos, Sui, CoreDAO, Sei** etc.

Contact

 **Website**
www.exvul.com

 **Twitter**
[@EXVULSEC](https://twitter.com/EXVULSEC)

 **Email**
contact@exvul.com

 **Github**
github.com/EXVUL-Sec